**Software for Systematics and Evolution**

*Systematic Biology* (2024), **0**, 0, *pp.* 1–37
doi:10.1093/sysbio/treeflow

# TreeFlow: Probabilistic Programming and Automatic Differentiation for Phylogenetics

Christiaan Swanepoel,[*,1,2] Mathieu Fourment,[3] Xiang Ji,[4] Hassan Nasif,[5] Marc A Suchard,[6,7,8] Frederick A Matsen IV,[5,9,10] Alexei J Drummond[1,11]

[1]Centre for Computational Evolution, The University of Auckland, Auckland, New Zealand;
[2]School of Computer Science, The University of Auckland, Auckland, New Zealand, 1010;
[3]Australian Institute for Microbiology and Infection, University of Technology Sydney, Ultimo NSW, Australia;
[4]Department of Mathematics, Tulane University, New Orleans, USA;
[5]Public Health Sciences Division, Fred Hutchinson Cancer Research Center, Seattle, Washington, USA;
[6]Department of Human Genetics, University of California, Los Angeles, USA;
[7]Department of Computational Medicine, University of California, Los Angeles, USA;
[8]Department of Biostatistics, University of California, Los Angeles, USA;
[9]Department of Statistics, University of Washington, Seattle, USA;
[10]Department of Genome Sciences, University of Washington, Seattle, USA;
[11]School of Biological Sciences, The University of Auckland, Auckland, New Zealand, 1010;

*Correspondence to be sent to: School of Computer Science, The University of Auckland, Auckland, New Zealand, 1010;*

*Email: cswa648@aucklanduni.ac.nz*

## Abstract

Probabilistic programming frameworks are powerful tools for statistical modelling and inference. They are not immediately generalisable to phylogenetic problems due to the particular computational properties of the phylogenetic tree object. TreeFlow is a software library for probabilistic programming and automatic differentiation with phylogenetic trees. It implements inference algorithms for node heights on phylogenetic trees and model parameters given a tree topology. We demonstrate how TreeFlow can be used to quickly implement and assess new models. We also show that it provides reasonable performance for gradient-based inference algorithms compared to specialized computational libraries for

© The Author 2024. Published by Oxford University Press. All rights reserved. For permissions, please e-mail: journals.permissions@oup.com

phylogenetics.

*Key words*: phylogenetics; Bayesian inference; probabilistic programming; variational inference

## Introduction

Traditionally, phylogenetic analyses have been performed by specialized software (Stamatakis, 2014; Drummond et al., 2012; Huelsenbeck and Ronquist, 2001). A number of software packages exist that implement a broad but predefined collection of models and associated specialized inference methods. Typically, inference is handled by carefully crafted but computationally costly stochastic optimisation or Markov chain Monte Carlo (MCMC) methods (Metropolis et al., 1953; Hastings, 1970). In contrast, in other realms of statistical analysis, *probabilistic programming* software libraries have entered into widespread use. These allow the specification of almost any model as a probabilistic program, and inference is provided automatically with a generic inference method. Exploiting the power of probabilistic programming in phylogenetic analyses could significantly accelerate research by making the process of developing new models and implementing inference faster and more flexible, as evidenced by recent work attempting to reconcile probabilistic programming frameworks with phylogenetic models (Fourment and Darling, 2019; Ronquist et al., 2021).

Probabilistic programming tools, such as BUGS (Lunn et al., 2000), Stan (Carpenter et al., 2017), PyMC3 (Salvatier et al., 2016), Pyro (Bingham et al., 2019), and TensorFlow Probability (Dillon et al., 2017) allow users to specify probabilistic models by describing the generative process with code as a *probabilistic program*. Advancements in automatic inference methods have allowed these tools to perform efficient inference on almost any model. Some notable examples, including automatic differentiation variational

inference (Kucukelbir et al., 2017) (ADVI) and Hamiltonian Monte Carlo (Duane et al., 1987) (HMC), use local gradient information from the model's probability density function to efficiently navigate the parameter space.

One key technology that enables these gradient-based automatic inference algorithms is *automatic differentiation*. Automatic differentiation refers to methods which efficiently calculate machine-precision gradients of functions, specified by computer programs, without extra analytical derivation or excessive computational overhead compared to the function evaluation. Automatic differentiation frameworks that have statistical inference packages built on top of them include Theano (Bergstra et al., 2010), Pytorch (Paszke et al., 2019), JAX (Bradbury et al., 2018) and TensorFlow (Abadi et al., 2016). Some of these extend to non-trivial computational constructs such as control flow and recursion (Yu et al., 2018).

The structure of the phylogenetic tree object is a major barrier to implementing efficient inference for phylogenetic probabilistic models. It is not clear how the association between its discrete and continuous quantities (the topology and branch lengths respectively) should be represented and handled in inference. Also, the combinatorial explosion of the size of discrete part of the phylogenetic state space presents a major challenge to any inference algorithm. Generic random search methods for discrete variables, as in the naive implementation of MCMC sampling, do not scale appropriately to allow inference on modern datasets with thousands of taxa.

Efforts have already been made to apply probabilistic programming to phylogenetic methods (Fourment and Darling, 2019; Ronquist et al., 2021; Drummond et al., 2022). RevBayes (Höhna et al., 2016) implements a probabilistic programming language that is focused on phylogenetic models. However, it requires the user to manually specify an MCMC scheme for performing inference. In other work, it has been shown that *universal probabilistic programming languages*, a particularly expressive extension of the class of traditional probabilistic programming languages, can be used to express generative

processes for trees, and use them to generate Sequential Monte Carlo inference schemes for complex speciation models (Ronquist et al., 2021). Blang (Bouchard-Côté et al., 2022) is a probabilistic modelling framework which supports arbitrary data structures that has been used to implement phylogenetic models and can automatically construct inference schemes. Another tool, LinguaPhylo, provides a domain specific modelling language for phylogenetics and has the capability to generate a specification for an MCMC sampler for inference (Drummond et al., 2022). These approaches both potentially lack the inherent scalability that the automatic inference methods that accompany general-purpose probabilistic modelling tools provide.

Applying scalable inference methods to phylogenetics is a major challenge. *Probabilistic path Hamiltonian Monte Carlo* has been developed to use gradient information to sample phylogenetic posteriors across multiple tree topologies (Dinh et al., 2017), though moves between tree topologies are not totally informed by gradient information and are similar to the random-walk proposal distributions available to standard MCMC routines. Hamiltonian Monte Carlo proposals for continuous parameters within tree topologies has been paired with standard MCMC moves between topologies for estimating local mutation rates and divergence times using scalable algorithms for gradient calculation (Ji et al., 2021; Fisher et al., 2021). Another approach has used Stan to apply automatic differentiation variational inference to phylogenetic inference within a fixed rooted tree topology (Fourment and Darling, 2019). Finally, variational inference has been applied to phylogenetic tree inference using a clade-based distribution on unrooted tree topologies (Zhang and Matsen IV, 2018). This approach has been extended to a more expressive family of approximating distributions (Zhang, 2020), and applied to rooted phylogenies (Zhang and Matsen IV, 2022).

One core computation that presents a challenge to automatic differentiation is the *phylogenetic likelihood* (Felsenstein, 1981). This computes the probability function of a collection of sequences given a phylogenetic tree, integrating out the character states at

ancestral nodes. This is most efficiently performed through dynamic programming, which requires sequential control flow or recursion based on the topology of the tree. When the tree topology is a dynamic variable the likelihood computation can be non-trivial to implement in a functional automatic differentiation framework such as TensorFlow. Additionally, naively computing the likelihood's gradient with respect to the branch lengths on the tree can yield poor computational scaling. Traversing the tree, as in a full likelihood evaluation, for the gradient computation for every branch would result in a quadratic computational complexity (Ji et al., 2020; Fourment et al., 2023).

## Software Description

TreeFlow is a Python library which provides modular tools for phylogenetic modelling and inference. It also provides a user interface which can be used for fixed-topology phylogenetic inference with models of a standard form. The architecture of the TreeFlow package is displayed in Figure 1.

The TreeFlow Python library includes modules for phylogenetic tree representation, traversal, and reading and writing. There are modules implementing bijections for time trees, phylogenetic posterior approximations, and fixed-topology variational Bayesian inference. It also implements tree priors and models of sequence evolution. The user interface implements a model definition format for standard phylogenetic models and command line interfaces for variational Bayesian and maximum likelihood inference.

It is important to note that the primary interface is intended to be the Python developer API, which allows users to implement new models and compose them with existing model components in a flexible structure, while leveraging the inferential machinery provided by TreeFlow. The user interface is provided to simplify testing and demonstration of TreeFlow, while also potentially making TreeFlow's scalable inference methods more accessible.

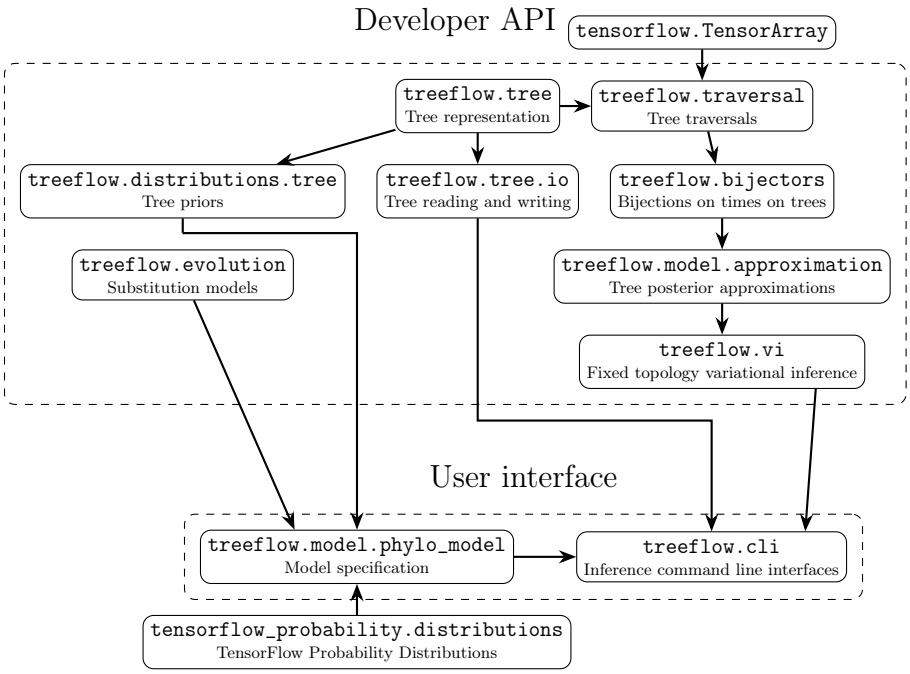


Fig. 1. Diagram showing the architecture of the TreeFlow package. Nodes represent Python modules and the arrows between them denote major dependencies. The dashed rectangles separate the components based on their primary interfaces. Nodes outside the two interface groups denote major external dependencies.

### *Tree representation*

TreeFlow is built on TensorFlow, a computational framework for machine learning. TreeFlow leverages TensorFlow's capabilities for accelerated numerical computation and automatic differentiation. TensorFlow's core computational object is the Tensor, a multi-dimensional array with a uniform data type. Phylogenetic trees are not immediately at home in a Tensor-centric universe, as they are a complex data structure, often defined recursively, with both continuous and discrete components. TreeFlow represents trees as a structure of Tensors. This structure contains floating point Tensors representing the times and branch lengths, and integer Tensors representing the topology, all encapsulated in a Python object. The topological Tensors include indices of node parents, children, and for pre- and post-order traversals. TensorFlow has extensive support for structures of Tensors such as the TreeFlow tree class, including using them as arguments to compiled functions and defining probability distributions over complex objects. This means computations and

models involving phylogenetic trees can be expressed naturally.

### *Tree distributions*

TreeFlow uses the existing probabilistic modelling infrastructure provided by TensorFlow Probability, which implements standard statistical distributions and inferential machinery (Dillon et al., 2017). The Distribution interface encapsulates functionality for sampling from a distribution and computing its probability density or mass function. TreeFlow implements a tree distribution base class which allows TensorFlow Probability Distributions to be defined on phylogenetic trees as represented by the tree objects described in the previous section.

TreeFlow implements Distributions for some commonly-used generative models for phylogenetic trees. The models currently implemented, Kingman's coalescent (Kuhner et al., 1995) and the Birth-Death-Sampling speciation process (Stadler, 2009), can be used as *tree priors* in phylogenetic probabilsitic models and to infer parameters related to population dynamics from genetic sequence data.

### *Models of sequence evolution*

The models of nucleotide sequence evolution currently implemented are the Jukes-Cantor (Jukes et al., 1969), HKY85 (Hasegawa et al., 1985), and General Time Reversible (GTR) (Tavaré et al., 1986) models. TreeFlow includes a standard approach for dealing with heterogeneity in mutation rate across sites based on a marginalizing over a discretized site rate distribution (Yang, 1994). The probabilistic modelling framework, however, allows for the use of any appropriate distribution as the base site rate distribution rather than just the standard single-parameter Gamma distribution. For example, it is straightforward to replace the base Gamma distribution with a Weibull distribution, which has a quantile function that is much easier to compute (Fourment and Darling, 2019). Thanks to TensorFlow's vectorized arithmetic operations, it is also natural

to model variations in mutation rate over lineages by specifying parameters for multiple rates (possibly with a hierarchical prior) and multiplying by the branch lengths of the phylogenetic (time) tree. Models which can be naturally expressed this way include the log-Normal random local clock (Drummond et al., 2006) and auto-correlated relaxed clock models (Thorne et al., 1998).

### *Tree traversals*

To perform model computations involving the phylogenetic tree, it is necessary to traverse the tree according to the structure specified by its topology. Since this structure is less accessible in an array-based tree representation, TreeFlow provides utilities to perform tree traversals. These are used to efficiently implement some core computations and also made accessible to developers for extending TreeFlow's functionality.

When iterating over the nodes of the tree in an automatic differentiation framework it is important to consider computational complexity. Since modern genomic datasets can result in the number of nodes in the tree being extremely large, it is important to maintain linear scaling with the size of the tree. Any computation that produces values for every node and requires access to results for multiple previous nodes in the traversal might incur a quadratic computational cost. This could occur when the topology is a dynamic variable as, at a given step of the iteration, values from any element of the Tensor representation might be need to be accessed. Since TensorFlow's automatic differentiation framework depends on the immutability of Tensors, an intermediate representation of the state for the whole tree needs to be maintained for every step of the iteration, leading to a quadratic complexity.

TreeFlow's solution to this scaling issue is to implement tree traversals using TensorFlow's `TensorArray` construct (Yu et al., 2018). The TensorArray is a data structure representing a collection of tensors. The TensorArray relaxes the requirement of immutability to a *write-once* property enforced on its constituent tensors. The

TensorArray tracks dependencies between values so automatic differentiation can appropriately propagate gradients back through computation in linear time.

A notable computation that makes use of the tree traversal framework described above is the *phylogenetic likelihood*, the probability of a sequence alignment given a phylogeny and model of sequence evolution. This typically involves integrating out character states at unsampled ancestral nodes using a *dynamic programming* computation known as *Felsenstein's pruning algorithm* (Felsenstein, 1981). Considerable attention has been given to the problem of efficiently computing gradients of branch lengths with respect to the phylogenetic likelihood (Ji et al., 2020). The dynamic programming algorithm is implemented as a TensorArray-based *postorder* traversal. The branch-length gradient computed through automatic differentiation demonstrates linear scaling with respect to the number of taxa in the tree in our benchmarks (see Benchmarks section). Our implementation also allows computation of gradients which respect to any substitution model parameters.

Another useful tool for phylogenetic inference implemented in TreeFlow is the *node height ratio transform* (Kishino et al., 2001). This has been used to infer times on phylogenetic trees using maximum likelihood in the PAML software package (Yang, 2007). The ratio transform parametrizes the internal node heights of a tree as the ratio between a node's height and its parent's height. The heights can be computed from the ratios in a pre-order tree traversal. This transformation has a triangular Jacobian matrix, which means computing the determinant required for change of variable of a probability density can be computed in linear time with respect to the number of internal node heights (Fourment and Darling, 2019). In combination with a log transformation of the root height and a logit transformation of the ratios, a multivariate distribution that takes real values can be transformed into a distribution on internal node heights of rooted phylogenies. This has been applied to Bayesian phylogenetic inference in the context of automatic differentiation variational inference (Fourment and Darling, 2019) and Hamiltonian Monte

Carlo (Ji et al., 2021). The ratio transform is implemented in TreeFlow as a TensorArray-based *preorder* traversal.

### *Bijectors on phylogenetic trees*

The ratio transform described above is wrapped into a TensorFlow Probability Bijector which provides a convenient interface for transforming real-valued distributions into phylogenetic tree distributions. The Bijector is an interface which represents a transformation of samples from a multivariate probability distribution. It requires implementation of the forward transformation and its inverse (which necessarily exists if the transformation is a bijection), as well as the determinant of the Jacobian matrix of the transformation. This can be used by TensorFlow Probability to perform change of variable, rewriting the probability density of the base distribution in the space the bijector transforms it to.

Other TensorFlow Probability Bijectors can be used to transform real-valued distributions into ratios. These can be chained with the ratio transform Bijector to create bijections from any real-valued distribution to a set of times corresponding to the node ages of a valid rooted phylogenetic tree.

Another Bijector implemented in TreeFlow is the transformation from a distribution on node heights to a TreeFlow tree object with a fixed tree topology. The resulting tree object can be used with phylogenetic models such as TreeFlow's tree priors and phylogenetic likelihood.

### *Variational inference*

One form of statistical inference for which the gradient computation is essential is variational Bayesian inference (Jordan et al., 1999). The goal of Bayesian inference is to characterise a *posterior distribution* which represents uncertainty over model parameters. Variational Bayesian inference achieves this by optimizing an approximation to the

posterior distribution which has more convenient analytical properties. Typically, the optimisation routine used in variational inference can scale to a larger number of model parameters than the random-walk sampling methods used by MCMC methods.

One concrete implementation of variational inference is *automatic differentiation variational inference* (ADVI) (Kucukelbir et al., 2017). ADVI can perform inference on a wide range of probabilistic graphical models composed of continuous variables. It automatically constructs an approximation to the posterior by transforming a convenient base distribution to respect the domains of the model's component distributions. It then optimizes this approximation using stochastic gradient methods (Robbins and Monro, 1951; Bottou, 2010). Typically, independent Normal distributions are used as the base distribution for computational convenience. This is known as the *mean field approximation*, and for posterior distributions that have significant correlation-structure, skew, multi-modality, or heavy tails, can introduce error in parameter estimation (Blei et al., 2017). Possible solutions to this approximation error include highly flexible variational approximations with large numbers of parameters (Rezende and Mohamed, 2015) or structured approximations that are informed by the form of the model (Ambrogioni et al., 2021).

TreeFlow leverages the stochastic gradient optimizators already implemented in TensorFlow and the variational inference objectives in TensorFlow Probability. The discrete topology element of phylogenetic trees is an obstacle in the usage of these algorithms, which are typically restricted to continuous latent variables. Often, the phylogenetic tree topology is not the latent variable of interest, and is not a significant source of uncertainty (Yang et al., 2000). This can be the case when divergence times or other substitution or speciation model parameters are the focus. In this scenario, useful results can be obtained by performing inference with a fixed tree topology, such as one obtained from fast maximum likelihood methods. This is the approach taken by the NextStrain platform (Hadfield et al., 2018), which uses the scalability afforded by a fixed tree topology to allow

large-scale rapid phylodynamic analysis of pathogen molecular sequence data.

While the currently implemented inference scheme in TreeFlow focuses on the fixed-topology paradigm, the rest of the library is not restricted in this way. The Tensor-based tree representation means that the topology is part of TensorFlow's computational graph. Additionally, computations such as the phylogenetic likelihood are implemented with support for a dynamic tree topology. This means that automatic-differentation based inference schemes that also infer the tree topology could naturally be implemented with TreeFlow providing model specification and likelihood evaluation functionality.

### *Model approximations*

TreeFlow implements posterior approximations for ADVI using TensorFlow Probability's bijector framework to transform a base distribution. Approximations are constructed automatically from probabilistic models in the form of TensorFlow Probability joint distribution objects. A base distribution, by default an uncorrelated Normal, is split into the variables of the joint distribution, and transformed to match the support of the model variables. The base distribution for the phylogenetic tree is transformed using a chain of bijectors into ratios, then divergence times, and finally into a TreeFlow tree object. Learnable parameters to optimize are introduced by adding an initial trainable transformation to the base distribution. For the mean field approximation, this is an affine elementwise shifting and scaling operation.

ADVI opens the door to using TensorFlow's neural network framework to implement deep-learning-based variants such as variational inference with *normalizing flows* (Rezende and Mohamed, 2015), which transform the base distribution through invertible trainable neural network layers to better approximate complex posterior distributions. The use of inverse autoregressive flows (Kingma et al., 2016) is implemented as potentially more expressive alternative to the mean field shift and scale operation, but

since its computational and memory requirements scale quadratically with the number of variables and therefore the number of taxa in the tree it is inappropriate for large phylogenetic analyses.

### *Model definition*

TensorFlow Probability distributions can be composed into a *probabilistic graphical model* using TensorFlow Probability's joint distribution functionality (Piponi et al., 2020). The code to specify a joint distribution provides a concise representation of the model used in a data analysis. The ability to implement phylogenetic models in this framework means that automatic inference algorithms implemented in TensorFlow can be leveraged.

Because many phylogenetic analyses use models of a similar structure, TreeFlow provides a format for concisely specifying this kind of model. A single unpartitioned multiple sequence alignment can be modelled with a single substitution model, tree prior, and associated parameter priors. The model specification is a nested dictionary, with keys at various levels corresponding to model components, their parameters, and distributions used as priors. This is transformed into a TensorFlow Probability joint distribution. The model definition structure can be serialized in the YAML markup format which provides a text-based model definition format. An example of this is shown in Figure 6. The substitution and tree models can be drawn from those described above, as well as parameter priors from a range of standard probability distributions implemented in TensorFlow Probability. This specification differs from the files used by other phylogenetic inference software (Drummond and Rambaut, 2007; Höhna et al., 2016) in that it does not include any description of inferential machinery and simply provides a terse description of the model.

### *Command line interfaces*

As well as a library for probabilistic programming with TensorFlow Probability, TreeFlow provides command line interfaces for fixed-topology inference. These allow inference on standard phylogenetic models such as those performed by specialized software. For inputs, they take a nucleotide sequence alignment in the FASTA or Nexus format, a tree topology in Newick format, and a TreeFlow model specification in YAML format. Command line interfaces are provided for both automatic differentiation variational inference and maximum a posteriori inference.

## Biological examples

We used TreeFlow to perform fixed-topology phylogenetic analyses of two datasets. The first is an alignment of 62 mitochondrial DNA sequences from carnivores (Suchard and Rambaut, 2009). The second is a dataset of 980 influenza sequences (Vaughan et al., 2014). In both datasets maximum likelihood unrooted topologies are estimated using RAxML (Stamatakis, 2014). These topologies are rooted with Least Squares Dating (To et al., 2016).

### *Probabilistic programming and model selection*

In the carnivores dataset, we demonstrate the flexibility of probabilistic programming with TreeFlow by investigating variation in the ratio of transitions to transversions in the nucleotide substitution process across lineages in the tree. Early maximum likelihood analyses of mitochondrial DNA in primates detected variation in this ratio, but without a biological basis, it was attributed to a saturation effect (Yang and Yoder, 1999). A later simulation-based investigation showed that this was a reasonable explanation, which exposed the limitations of nucleotide substitution models for estimating the length of branches deep in the tree (Duchêne et al., 2015).

This problem could be approached by means of Bayesian model comparison; a lack

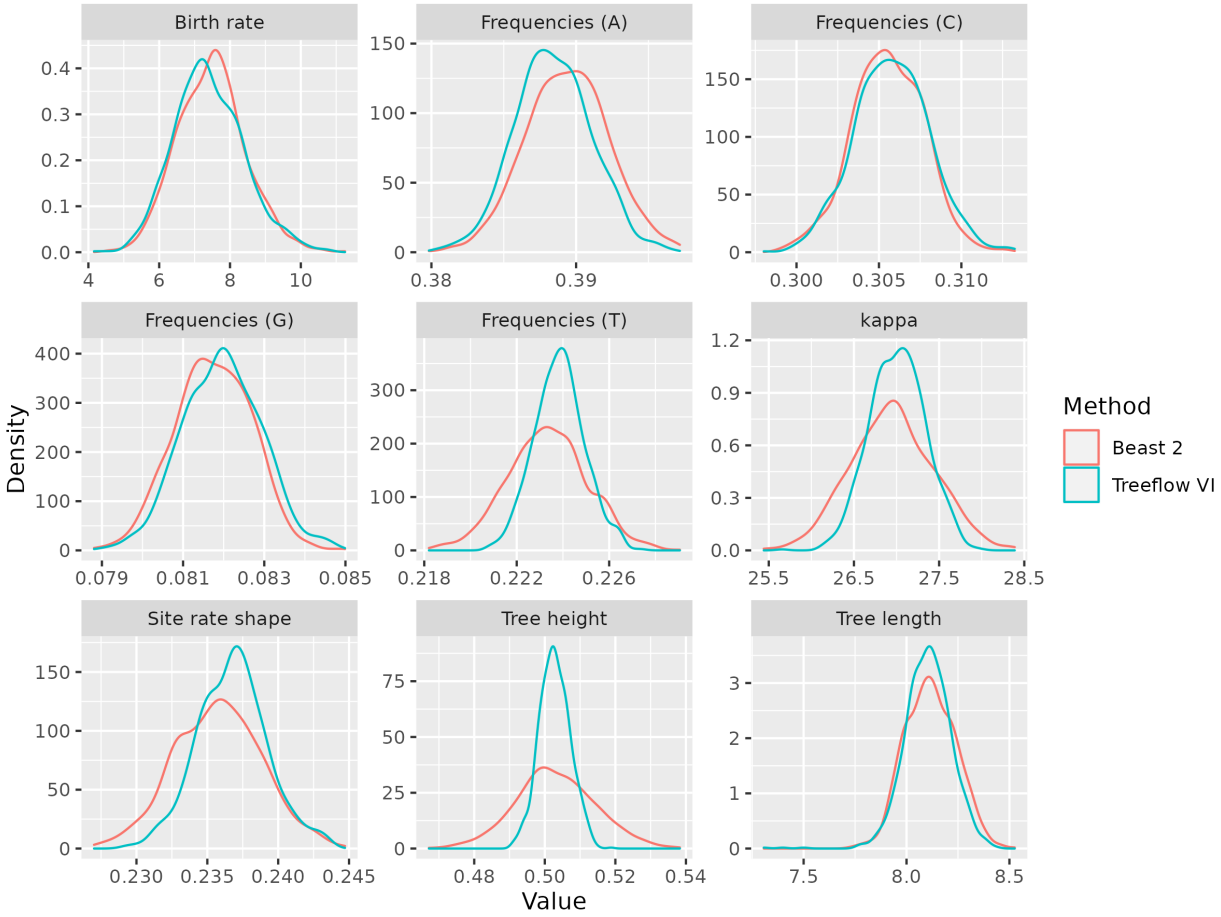


Fig. 2. Carnivores base model posterior marginal parameter estimates

of preference for a model allowing between-lineage variation of the ratio could indicate that the substitution model lacks the power to separate variation 'signal' from saturation 'noise'. Firstly, we construct a standard phylogenetic model with a HKY substitution model with a single transition-transversion ratio parameter (*kappa*). We perform inference using ADVI, and also using MCMC as implemented in BEAST 2 (Bouckaert et al., 2019). Since this model is of the form of a standard phylogenetic analysis, it could be fit using TreeFlow's command line interface. Figure 2 compares the marginal parameter estimates obtained from TreeFlow and BEAST 2. The discrepancies in distribution, apparent in the estimates of the frequencies and tree height, can be attributed to the approximation error

introduced by ADVI. Most importantly, the estimate of the parameter of interest, kappa, appears reasonable.

We then altered this model to estimate a separate ratio for every lineage on the tree; the implementation of this was simple and scalable as a result of TensorFlow's vectorization and broadcasting functionality. Figure 3 shows the model definition code for the base model, and the small changes required to model variation in the kappa parameter across lineages. We compared the models using estimates of the marginal likelihood (MacKay et al., 2003). The *marginal likelihood*, or *evidence*, the integral of the likelihood of the data over model parameters, is typically analytically intractable and challenging to compute using MCMC methods (Xie et al., 2011). The closed-form approximation to the posterior distribution provided by variational inference means we could estimate the marginal likelihood using importance sampling, a utility provided by TreeFlow. This corrects for some of the posterior approximation error described above.

The estimated per-lineage kappa parameters are shown in Figure 4. Kappa estimates decline with increasing age of the lineage, which agrees with the results of the previous studies. The marginal likelihood for the model with transition-transversion ratio variation across lineages was higher than for the base model. This means that, under the other components of this model, the data supports variation in the ratio. This does not necessarily imply that transition-transversion ratio in the true generative process of the data varies in the same way, but could be a useful consideration in designing more sophisticated models of nucleotide substitution. The growth in uncertainty in kappa estimates deeper in the tree supports previous conclusions that nucleotide substitution models are unable to effectively estimate the number of substitutions on older branches (Duchêne et al., 2015). Figure 5 shows that the kappa variation model shortens older branches, leading to a substantially reduced overall tree height estimate, with proportionally similar uncertainty in node height estimates. This is not a proper dating analysis as it does not consider uncertainty in the mutation rate and is not time-calibrated,

```
site_category_count = 4
pattern_counts = alignment.get_weights_tensor()
subst_model = HKY()

def build_sequence_dist(tree, kappa, frequencies, site_gamma_shape):
    unrooted_tree = tree.get_unrooted_tree()
    site_rate_distribution = DiscretizedDistribution(
        category_count=site_category_count,
        distribution=Gamma(
            concentration=site_gamma_shape,
            rate=site_gamma_shape
        ),
    )
    transition_probs_tree = get_transition_probabilities_tree(
        unrooted_tree,
        subst_model,
        rate_categories=site_rate_distribution.normalised_support,
        frequencies=frequencies,
        frequencies=tf.broadcast_to(
            tf.expand_dims(frequencies, -2),
            kappa.shape + (4,)
        ),
        kappa=kappa
    )
    return SampleWeighted(
        DiscreteParameterMixture(
            site_rate_distribution,
            LeafCTMC(
                transition_probs_tree,
                expand_dims(frequencies, -2),
            ),
        ),
        sample_shape=alignment.site_count,
        weights=pattern_counts
    )

model = JointDistributionNamed(dict(
    birth_rate=LogNormal(c(1.0), c(1.5)),
    tree=lambda birth_rate: Yule(tree.taxon_count, birth_rate),
    kappa=LogNormal(c(0.0), c(2.0)),
    kappa=Sample(
        LogNormal(c(0.0), c(2.0)),
        tree.branch_lengths.shape
    ),
    site_gamma_shape=LogNormal(c(0.0), c(1.0)),
    frequencies=Dirichlet(c([2.0, 2.0, 2.0, 2.0])),
    sequences=build_sequence_dist
))
```

Fig. 3. Code to specify models for carnivores analysis. Highlighted lines show changes (from previous line) to add kappa variation across lineages.

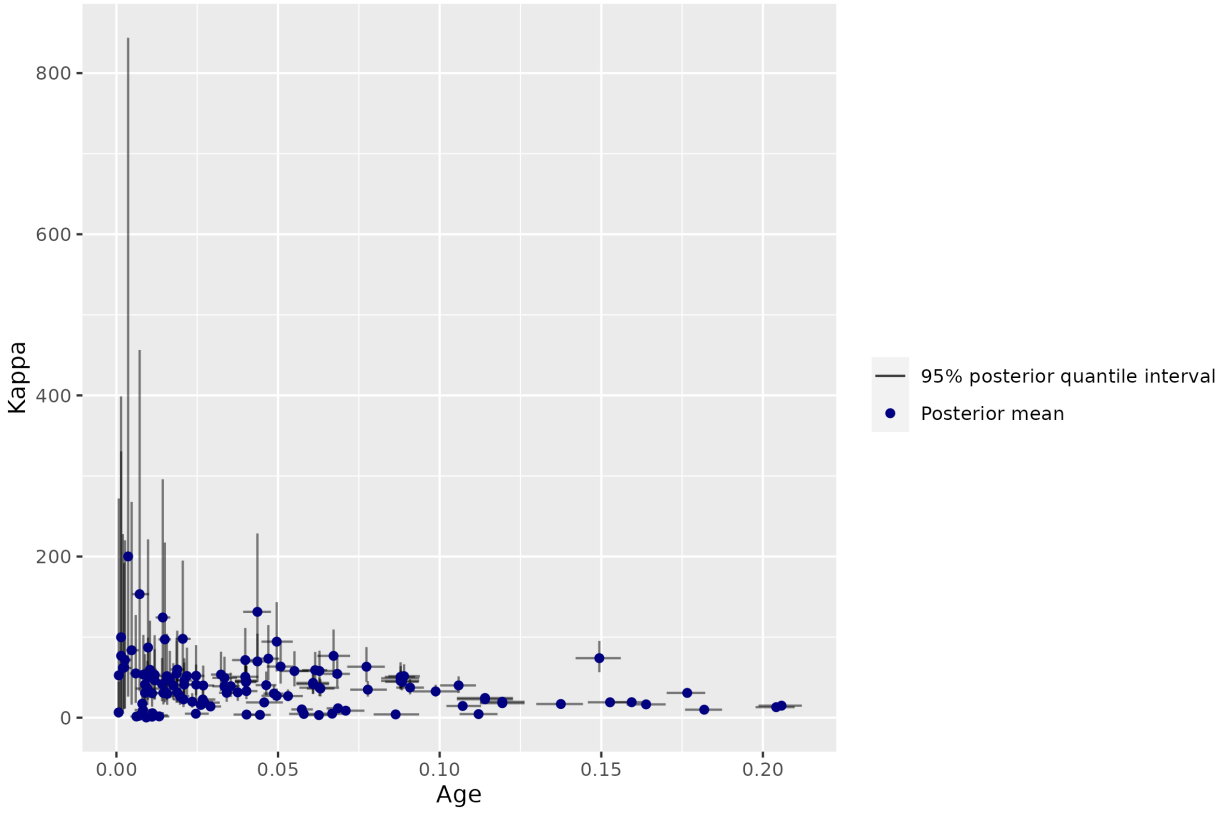


Fig. 4. Lineage age (number of expected substitutions per site before the present) vs estimated transition/tranversion ratio (Kappa) for carnivores dataset

but it is clear that this model could significantly affect time estimates.

### *Scalable inference*

In the analysis of the 980-taxon influenza dataset, we demonstrate the scalability of variational inference. We performed variational inference using TreeFlow's command line interface. The model used for inference included a coalescent prior with a constant population size on the tree, a strict molecular clock, a discretized Gamma model of site rate variation and a GTR substitution model. The YAML model definition code for this model, including parameter priors, is shown in Figure 6. This parameterization of the GTR substitution model (here named `gtr_rel`), with independent priors on five of the relative rates and one held fixed to 1, is used to allow comparison of parameter estimates with

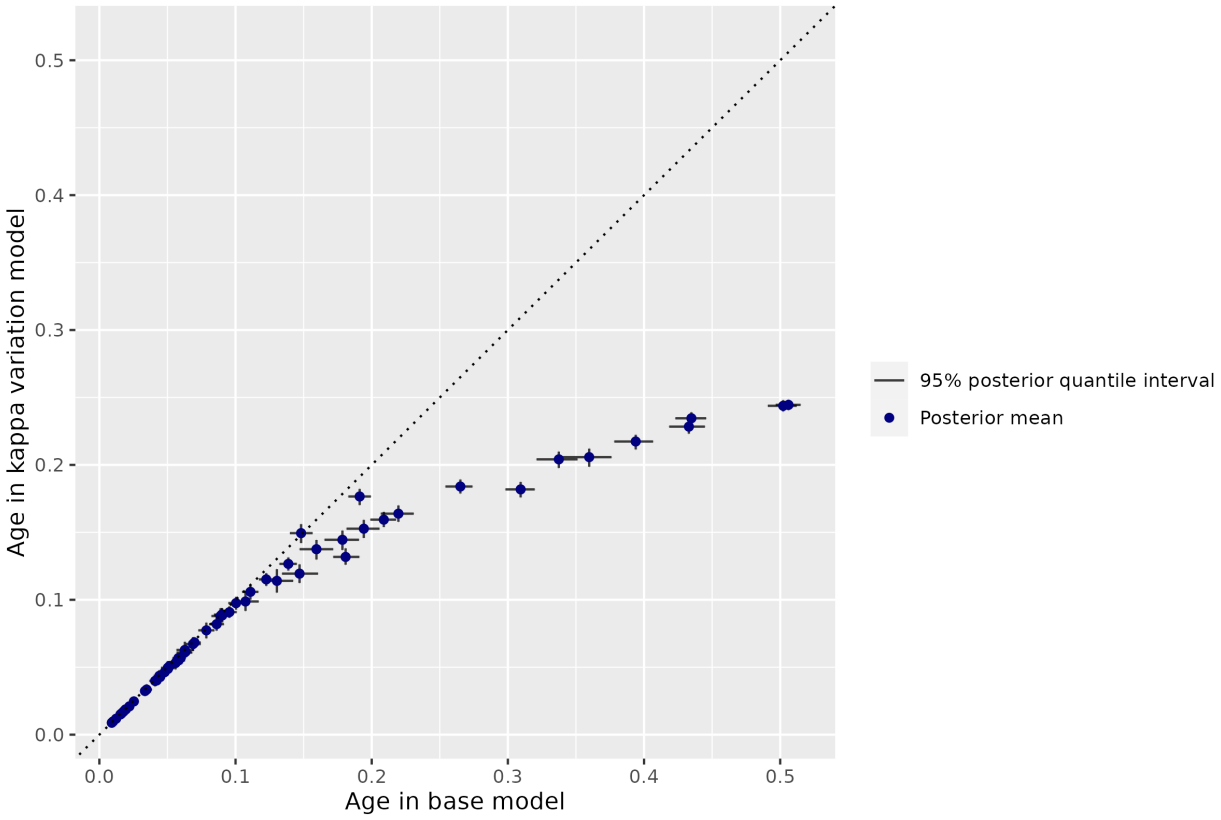


Fig. 5. Node age estimates for base model vs per-lineage kappa variation model

BEAST 2. It is also possible to use a six-rate GTR parameterisation with a Dirichlet prior in TreeFlow.

Figure 7 shows the marginal parameter estimates obtained from this dataset, compared to those obtained from a fixed-topology MCMC analysis using BEAST 2. The posterior distributions of most parameters are approximated with high fidelity by the mean field approximation. The uncertainty in the tree height, coalescent effective population size, and clock rate are slightly underestimated. This is likely a result of using an approximation that ignores the correlations between these parameters that are present in the posterior. The objective function used in ADVI, the KL divergence between the approximating and posterior distributions, heavily penalises regions of the approximation where there is no posterior support, which typically results in low uncertainty estimates.

```yaml
clock:
  strict:
    clock_rate:
      lognormal:
        loc: -2.0
        scale: 2.0
site:
  discrete_gamma:
    category_count: 4
    site_gamma_shape:
      lognormal:
        loc: 0.0
        scale: 1.0
substitution:
  gtr_rel:
    frequencies:
      dirichlet:
        concentration:
        - 2.0
        - 2.0
        - 2.0
        - 2.0
    rate_ac:
      gamma:
        concentration: 0.05
        rate: 0.05
    rate_ag:
      gamma:
        concentration: 0.05
        rate: 0.05
    rate_at:
      gamma:
        concentration: 0.05
        rate: 0.05
    rate_cg:
      gamma:
        concentration: 0.05
        rate: 0.05
    rate_gt:
      gamma:
        concentration: 0.05
        rate: 0.05
tree:
  coalescent:
    pop_size:
      lognormal:
        loc: 1.0
        scale: 1.5
```

Fig. 6. YAML model definition for the analysis of the influenza dataset

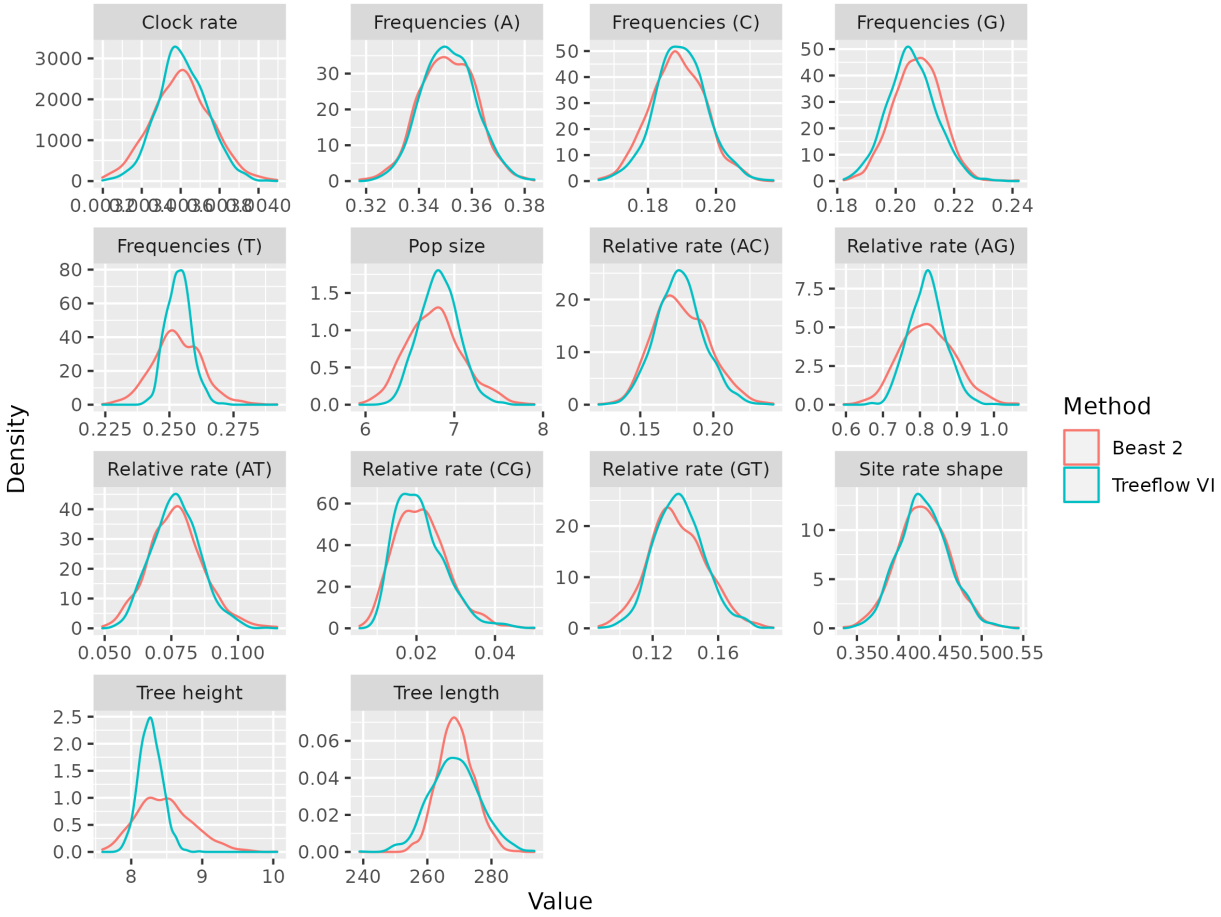


Fig. 7. Influenza parameter marginal posterior distributions

Figure 8 compares the divergence time estimates. TreeFlow's mean field variational approximation assumes the posterior distribution on the times of the tree are independent Normal distributions transformed through the node height ratio transformation to respect the time constraints of the tree. However, the true posterior is almost certainly not of this form, so the approximation introduces error. In general, the mean of the posterior distribution is well approximated, in particular for the oldest seven nodes of the tree. The posterior means of other divergence times are generally close but not identical to the true posterior. The error is more apparent in the posterior uncertainty; the standard deviation estimates produced by variational inference often differ substantially from the posterior

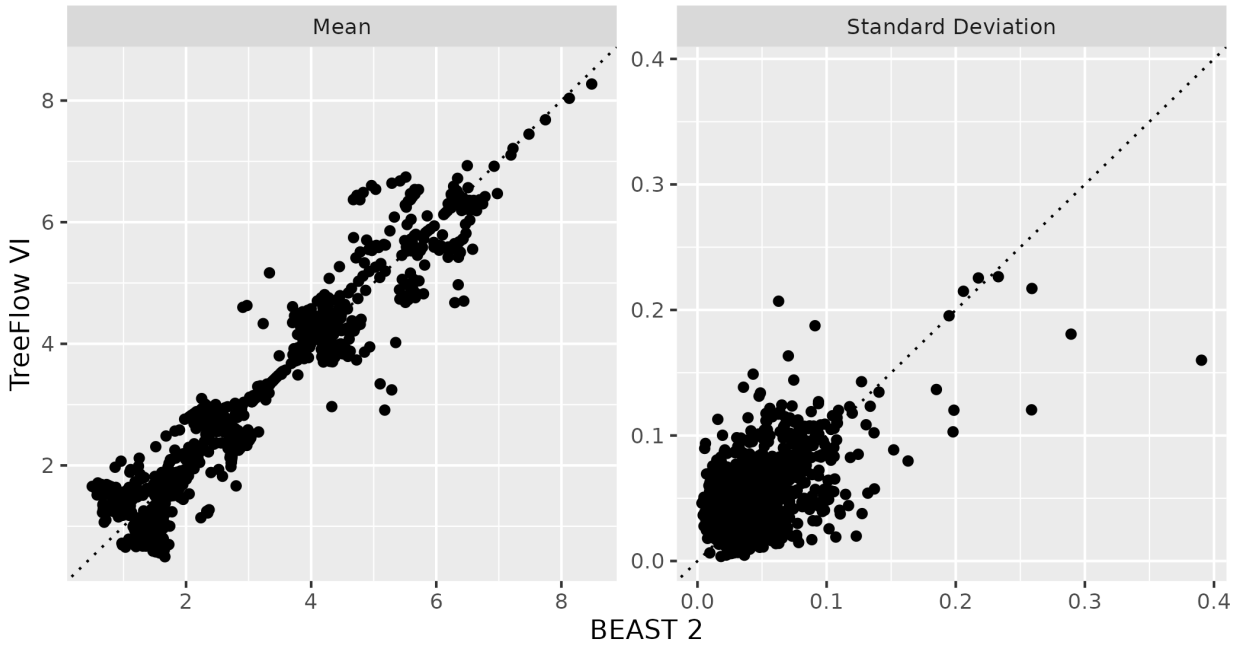


Fig. 8. Influenza node height posterior distribution statistics

estimated by BEAST 2. While mean field variational inference seems effective for estimating the parameters of the phylogenetic model the estimates of divergence times appear less reliable. Variational approximations that better capture the form of the true posterior would improve the quality of these estimates.

BEAST 2 MCMC sampling was run for 30,000,000 iterations. Convergence was checked using *effective sample size* (ESS), which was computed for all parameters using ArviZ (Kumar et al., 2019). Multiple runs were performed to tune MCMC operator weights to improve ESSs. Generally, the minimum acceptable ESS is 100, though 200 is preferred (Drummond and Bouckaert, 2015). The analysis resulted in a minimum effective sample size of 125. The wall clock runtime for the BEAST 2 analysis was 5 hours and 57 minutes. The TreeFlow analysis converged in 20,000 iterations which took 3 hours and 3 minutes. This shows that variational inference has favourable scaling for large phylogenetic datasets despite the more expensive gradient computations required, and that TreeFlow's slower phylogenetic likelihood implementation performs well enough to be useful in

real-world inference scenarios.

## Benchmarks

The phylogenetic likelihood is the computation that dominates the computational cost of model-based phylogenetic inference. We benchmark the performance of our TensorFlow-based likelihood implementation against specialized libraries. Since gradient computations are of equal importance to likelihood evaluations in modern automatic inference regimes, we also benchmark computation of derivatives of the phylogenetic likelihood, with respect to both the continuous elements of the tree and the parameters of the substitution model. A clear difference emerges in the implementation of derivatives; TreeFlow's are based on automatic differentiation while bespoke libraries need analytically-derived gradients. Therefore, we do not necessarily expect our implementation to be as fast as bespoke software, but it does not rely on analytical derivation of gradient expressions for every model and therefore automatically supports a wider range of models.

We compare the performance of TreeFlow's likelihood implementation against BEAGLE (Ayres et al., 2019). BEAGLE is a library for high-performance computation of phylogenetic likelihoods. Recent versions of BEAGLE implement analytical gradients with respect to tree branch lengths (Ji et al., 2020). We use BEAGLE via the `bito` software package (Matsen et al.), which provides a Python interface to BEAGLE and also numerically computes derivatives with respect to substitution model parameters using a finite differences approximation.

We also compare TreeFlow with a simple likelihood implementation (Swanepoel) based on another automatic differentiation framework, JAX (Bradbury et al., 2018). In contrast to TensorFlow's function mode, which the benchmarked TreeFlow implementation uses, JAX uses an eager execution model and is compatible with native Python control flow.

Benchmarks are performed on simulated data. Simulation allows the generation of a

large number of data replicates with appropriate properties. Since we want to investigate the scaling of likelihood and gradient calculations with respect to the number of sequences, we simulate data for a range of sequence counts. Sequence counts are selected as increasing powers of 2 to better display asymptotic properties of the implementations. We simulate data with sequence counts ranging from 32 to 2048. Trees are simulated under a coalescent model for a given number of taxa, and then nucleotide sequences of length 1000 are simulated under a fixed rate of evolution and a HKY substitution model. Tree and sequence simulations are performed using BEAST 2 (Bouckaert et al., 2019).

We benchmark 4 distinct computations on these datasets. Firstly, for each replicate we compute the phylogenetic likelihood under a very simple model of sequence evolution. This uses a Jukes-Cantor model of nucleotide substitution and no model of site rate variation. Secondly, we calculate derivatives with respect to branch lengths under this simple model. Thirdly, we compute the likelihood under a more sophisticated substitution model, with a discretized Weibull distribution with 4 rate categories to model site rate variation and a GTR model of nucleotide substitution. We selected the Weibull distribution for site rate variation since it is implemented in `bito`. Finally, we compute derivatives with respect to both branch lengths and the parameters of the substitution model (the 6 GTR relative substitution rates, 4 base nucleotide frequencies, and the Weibull site rate shape parameter). Each computation is performed 100 times on 10 simulated datasets.

Figure 9 shows the results of the benchmarks with a log scale on both axes. For likelihood computation with both models and branch gradient computation with the simple model `bito`/BEAGLE are at least an order of magnitude faster than TreeFlow. This is expected as BEAGLE is a highly optimized special-purpose library written in native code. The performance gap grows much smaller for computing the gradients of the more complex substitution model. `bito` performs at least 2 likelihood evaluations for each additional parameter when calculating the gradient with respect to substitution model parameters, while the overhead for substitution model parameters with automatic

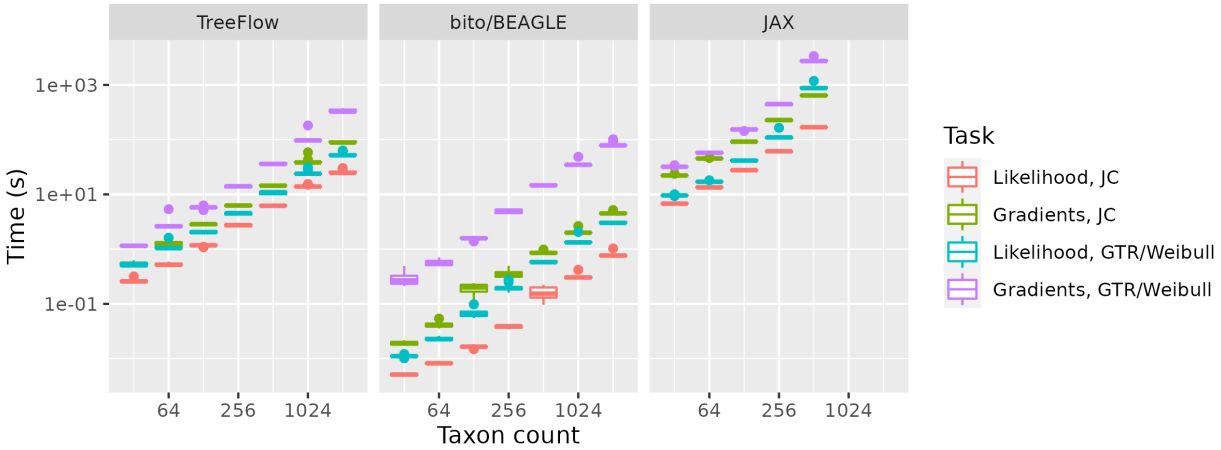


Fig. 9. Times from phylogenetic likelihood benchmark (100 evaluations for 1000 sites)

| Model | Computation | Method | Mean time (512 taxa) | log-log slope |
|---|---|---|---|---|
| JC | Likelihood | TreeFlow | 6.23 | 1.13 |
| | | bito/BEAGLE | 0.16 | 1.27 |
| | | JAX | 168.79 | 1.15 |
| | Gradients | TreeFlow | 14.55 | 1.23 |
| | | bito/BEAGLE | 0.87 | 1.33 |
| | | JAX | 644.22 | 1.19 |
| GTR/Weibull | Likelihood | TreeFlow | 10.73 | 1.12 |
| | | bito/BEAGLE | 0.58 | 1.40 |
| | | JAX | 908.24 | 1.58 |
| | Gradients | TreeFlow | **36.14** | **1.34** |
| | | bito/BEAGLE | **14.74** | **1.41** |
| | | JAX | **2801.35** | **1.58** |

Table 1. Results of phylogenetic likelihood benchmark. Times for gradients for the GTR/Weibull are highlighted as they are the most relevant computation for gradient-based inference on real data.

differentiation is minimal. We expect TreeFlow to surpass `bito`/BEAGLE for substitution models with even more parameters (e.g. amino acid substitution models (Adachi and Hasegawa, 1996)), or those where the number of parameters grows with the number of sequences, as in the example using the carnivore dataset above.

For typical real-world phylogenetic analyses, a model with many parameters, such as the GTR/Weibull model in our benchmarks, is used. For modern Bayesian inference

methods such as variational inference and Hamiltonian Monte Carlo, the gradient is the primary computation performed at each iteration. We observe that TreeFlow's times for this combination of model and computation are within an order of magnitude of `bito`/BEAGLE's, around 2-3 times slower. Therefore for applied analyses the automatic differentiation-based computations of TreeFlow presents a reasonable alternative to specialized software while similtaneously offering greater flexibility.

We also observe that the runtimes for TreeFlow are roughly an order of magnitude less than those of the JAX-based implementation. These indicate that the control flow constructs and execution model of TensorFlow are a good choice for implementing tree-based computations compared to the eager execution model of JAX.

Table 1 shows the coefficients obtained from fitting a linear model to the benchmark times where the predictor is the log-transformed number of sequences and the target is the log-transformed runtime. The slope parameter estimates from these fits is a rough empirical estimate of the polynomial degree of the computational scaling. The slope parameters for all TreeFlow computations are well below 2 and indicate a roughly linear scaling with the size of the data. For the JC benchmark, the only gradients computed are with respect to branch lengths, so the whole computation for `bito`/BEAGLE is the analytical linear-time branch gradient. TreeFlow's scaling is certainly not worse in this case, indicating that the TensorArray-based likelihood implementation enables linear-time branch gradients.

## Discussion

The combination of flexibility and scalability of libraries like TreeFlow has the potential to be useful for rapid analysis of large modern genetic datasets, such as those assembled for tracking infectious diseases (Hadfield et al., 2018). For this purpose, TreeFlow's true power would be unlocked with the implementation of improved modelling and inference tools for evolutionary rates and population dynamics.

For principled dating analyses, uncertainty in evolutionary rate parameters must be

considered. Posterior distributions involving these parameters may have significant correlation structure which would be ignored by variational inference using a mean-field posterior approximation. These parameters typically receive special treatment in MCMC sampling routines (Drummond et al., 2006; Zhang and Drummond, 2020). As observed in our analysis on the influenza dataset, the mean field approximation fails to accurately capture the posterior distribution on all internal divergence time estimates. Implementing structured posterior approximations for these models could substantially improve the reliability of parameter estimates obtained using variational inference while scaling to extremely large datasets.

Additionally, implementing flexible models for population dynamics, such as nonparametric coalescent models (Drummond et al., 2005; Minin et al., 2008; Gill et al., 2013), could provide valuable insights from large datasets. These models would result in complex posterior distributions, which would need to be accounted for in a variational inference scheme. If these coalescent models could be made to work effectively in TreeFlow, the probabilistic programming paradigm would allow for rapid computational experimentation with novel models of time-varying population parameters.

Finally, the most significant functionality for phylogenetic inference missing from TreeFlow is inference of the tree topology. TreeFlow's tree representation could allow for the topology to become an estimated parameter, such as in existing work on variational Bayesian phylogenetic inference (Zhang and Matsen IV, 2018). Efficiently implementing the computations required for these algorithms in the TensorFlow framework would certainly be a major computational challenge, but would be supported by TreeFlow's representation of the tree topology as a dynamic variable inside the computational graph.

The probabilistic modelling approach enabled by the TreeFlow Python API in conjunction with TensorFlow Probability has similar goals to other phylogenetic probabilistic modelling software. Two notable examples, RevBayes (Höhna et al., 2016) and Blang (Bouchard-Côté et al., 2022) provide a generic model definition and inference

engine which also support phylogenetic models and tree inference. Some notable differences exist between TreeFlow and these which may prove advantageous for certain users. Firstly, RevBayes requires the user to manually specify an MCMC operator scheme, including parameters and weights. This may require substantial user time and expertise to achieve correct and efficient inference. Blang and TreeFlow, on the other hand, are focused on automatic inference methods that require less user input. Secondly, TreeFlow and it's Python model specification API are implemented in a widely used computational framework (TensorFlow) and programming language (Python). As a result it can potentially leverage other computational tools implemented in TensorFlow, including less common distributions, machine learning models, differential equation solvers and alternative inference schemes. The automatic differentiation capabilities TensorFlow affords mean TreeFlow can interface with the large and growing collection of models that rely on gradient-based inference. On the other hand, the choice of computational framework incurs a performance cost as evidenced by the results of benchmarking. Blang models can make use of external Java code, but the ecosystem of related software in Java is less developed, and RevBayes does not provide any functionality for interfacing with software that is not implemented in the Rev language.

## Conclusion

TreeFlow is a software library that provides tools for statistical modelling and inference on phylogenetic problems. Probabilistic programming provides flexible model definition involving phylogenetic trees and molecular sequences, and automatic differentiation enables the application of modern scalable inference algorithms. TreeFlow's automatic differentiation-based implementations of core gradient computations provide reasonable performance compared to specialized phylogenetic libraries. These building blocks have the potential to be used in valuable phylogenetic analyses of large and complex genetic datasets.

## Availability

TreeFlow is an open-source Python package. Instructions for installing TreeFlow, and examples of using it as a library and command line application can be found at `https://github.com/christiaanjs/treeflow`.

## Supplementary material

Datasets, model definitions, starting values, and BEAST 2 XML files used in the biological examples are available at `https://github.com/christiaanjs/treeflow-paper/releases/tag/v0.0.1`.

## Conflicts of Interest

The authors declare that there are no conflicts of interest.

## References

Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. TensorFlow: A system for large-scale machine learning. In *12th USENIX symposium on operating systems design and implementation (OSDI 16)*, pages 265–283, 2016.

Jun Adachi and Masami Hasegawa. Model of amino acid substitution in proteins encoded by mitochondrial DNA. *Journal of Molecular Evolution*, 42(4):459–468, 1996.

Luca Ambrogioni, Kate Lin, Emily Fertig, Sharad Vikram, Max Hinne, Dave Moore, and Marcel van Gerven. Automatic structured variational inference. In *International Conference on Artificial Intelligence and Statistics*, pages 676–684. PMLR, 2021.

Daniel L Ayres, Michael P Cummings, Guy Baele, Aaron E Darling, Paul O Lewis, David L Swofford, John P Huelsenbeck, Philippe Lemey, Andrew Rambaut, and Marc A

Suchard. BEAGLE 3: Improved performance, scaling, and usability for a high-performance computing library for statistical phylogenetics. *Systematic Biology*, 68 (6):1052–1061, 2019.

James Bergstra, Olivier Breuleux, Frédéric Bastien, Pascal Lamblin, Razvan Pascanu, Guillaume Desjardins, Joseph Turian, David Warde-Farley, and Yoshua Bengio. Theano: a CPU and GPU math expression compiler. In *Proceedings of the 9th Python in Science Conference*, pages 1–7, 2010.

Eli Bingham, Jonathan P Chen, Martin Jankowiak, Fritz Obermeyer, Neeraj Pradhan, Theofanis Karaletsos, Rohit Singh, Paul Szerlip, Paul Horsfall, and Noah D Goodman. Pyro: Deep universal probabilistic programming. *The Journal of Machine Learning Research*, 20(1):973–978, 2019.

David M Blei, Alp Kucukelbir, and Jon D McAuliffe. Variational inference: A review for statisticians. *Journal of the American Statistical Association*, 112(518):859–877, 2017.

Léon Bottou. Large-scale machine learning with stochastic gradient descent. In *Proceedings of COMPSTAT'2010*, pages 177–186. Springer, 2010.

Alexandre Bouchard-Côté, Kevin Chern, Davor Cubranic, Sahand Hosseini, Justin Hume, Matteo Lepur, Zihui Ouyang, and Giorgio Sgarbi. Blang: Bayesian declarative modeling of general data structures and inference via algorithms based on distribution continua. *Journal of Statistical Software*, 103:1–98, 2022.

Remco Bouckaert, Timothy G Vaughan, Joëlle Barido-Sottani, Sebastián Duchêne, Mathieu Fourment, Alexandra Gavryushkina, Joseph Heled, Graham Jones, Denise Kühnert, Nicola De Maio, et al. BEAST 2.5: An advanced software platform for Bayesian evolutionary analysis. *PLoS computational biology*, 15(4):e1006650, 2019.

James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye

Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL `http://github.com/google/jax`.

Bob Carpenter, Andrew Gelman, Matthew D Hoffman, Daniel Lee, Ben Goodrich, Michael Betancourt, Marcus Brubaker, Jiqiang Guo, Peter Li, and Allen Riddell. Stan: A probabilistic programming language. *Journal of Statistical Software*, 76(1), 2017.

Joshua V Dillon, Ian Langmore, Dustin Tran, Eugene Brevdo, Srinivas Vasudevan, Dave Moore, Brian Patton, Alex Alemi, Matt Hoffman, and Rif A Saurous. TensorFlow distributions. *arXiv preprint arXiv:1711.10604*, 2017.

Vu Dinh, Arman Bilge, Cheng Zhang, and Frederick A Matsen IV. Probabilistic path Hamiltonian Monte Carlo. In *International Conference on Machine Learning*, pages 1009–1018. PMLR, 2017.

Alexei J Drummond and Remco R Bouckaert. *Bayesian evolutionary analysis with BEAST*. Cambridge University Press, 2015.

Alexei J Drummond and Andrew Rambaut. BEAST: Bayesian evolutionary analysis by sampling trees. *BMC Evolutionary Biology*, 7(1):1–8, 2007.

Alexei J Drummond, Andrew Rambaut, BETH Shapiro, and Oliver G Pybus. Bayesian coalescent inference of past population dynamics from molecular sequences. *Molecular Biology and Evolution*, 22(5):1185–1192, 2005.

Alexei J Drummond, Simon YW Ho, Matthew J Phillips, and Andrew Rambaut. Relaxed phylogenetics and dating with confidence. *PLoS Biology*, 4(5):e88, 2006.

Alexei J Drummond, Marc A Suchard, Dong Xie, and Andrew Rambaut. Bayesian phylogenetics with BEAUti and the BEAST 1.7. *Molecular Biology and Evolution*, 29 (8):1969–1973, 2012.

Alexei J Drummond, Kylie Chen, Fábio K Mendes, and Dong Xie. LinguaPhylo: a

probabilistic model specification language for reproducible phylogenetic analyses. *bioRxiv*, 2022.

Simon Duane, Anthony D Kennedy, Brian J Pendleton, and Duncan Roweth. Hybrid Monte Carlo. *Physics Letters B*, 195(2):216–222, 1987.

Sebastián Duchêne, Simon YW Ho, and Edward C Holmes. Declining transition/transversion ratios through time reveal limitations to the accuracy of nucleotide substitution models. *BMC Evolutionary Biology*, 15(1):1–10, 2015.

Joseph Felsenstein. Evolutionary trees from DNA sequences: a maximum likelihood approach. *Journal of Molecular Evolution*, 17(6):368–376, 1981.

Alexander A Fisher, Xiang Ji, Akihiko Nishimura, Philippe Lemey, and Marc A Suchard. Shrinkage-based random local clocks with scalable inference. *arXiv preprint arXiv:2105.07119*, 2021.

Mathieu Fourment and Aaron E Darling. Evaluating probabilistic programming and fast variational Bayesian inference in phylogenetics. *PeerJ*, 7:e8272, 2019.

Mathieu Fourment, Christiaan J Swanepoel, Jared G Galloway, Xiang Ji, Karthik Gangavarapu, Marc A Suchard, and Frederick A Matsen Iv. Automatic differentiation is no panacea for phylogenetic gradient computation. *Genome biology and evolution*, 15(6): evad099, 2023.

Mandev S Gill, Philippe Lemey, Nuno R Faria, Andrew Rambaut, Beth Shapiro, and Marc A Suchard. Improving bayesian population dynamics inference: a coalescent-based model for multiple loci. *Molecular Biology and Evolution*, 30(3):713–724, 2013.

James Hadfield, Colin Megill, Sidney M Bell, John Huddleston, Barney Potter, Charlton Callender, Pavel Sagulenko, Trevor Bedford, and Richard A Neher. Nextstrain: real-time tracking of pathogen evolution. *Bioinformatics*, 34(23):4121–4123, 2018.

Masami Hasegawa, Hirohisa Kishino, and Taka-aki Yano. Dating of the human-ape splitting by a molecular clock of mitochondrial DNA. *Journal of Molecular Evolution*, 22 (2):160–174, 1985.

W. K. Hastings. Monte Carlo sampling methods using Markov chains and their applications. *Biometrika*, 57(1):97–109, 04 1970.

Sebastian Höhna, Michael J Landis, Tracy A Heath, Bastien Boussau, Nicolas Lartillot, Brian R Moore, John P Huelsenbeck, and Fredrik Ronquist. RevBayes: Bayesian phylogenetic inference using graphical models and an interactive model-specification language. *Systematic Biology*, 65(4):726–736, 2016.

John P Huelsenbeck and Fredrik Ronquist. MRBAYES: Bayesian inference of phylogenetic trees. *Bioinformatics*, 17(8):754–755, 2001.

Xiang Ji, Zhenyu Zhang, Andrew Holbrook, Akihiko Nishimura, Guy Baele, Andrew Rambaut, Philippe Lemey, and Marc A Suchard. Gradients do grow on trees: a linear-time O(N)-dimensional gradient for statistical phylogenetics. *Molecular Biology and Evolution*, 37(10):3047–3060, 2020.

Xiang Ji, Alexander A Fisher, Shuo Su, Jeffrey L Thorne, Barney Potter, Philippe Lemey, Guy Baele, and Marc A Suchard. Scalable bayesian divergence time estimation with ratio transformations. *arXiv preprint arXiv:2110.13298*, 2021.

Michael I Jordan, Zoubin Ghahramani, Tommi S Jaakkola, and Lawrence K Saul. An introduction to variational methods for graphical models. *Machine Learning*, 37(2): 183–233, 1999.

Thomas H Jukes, Charles R Cantor, et al. Evolution of protein molecules. *Mammalian protein metabolism*, 3:21–132, 1969.

Durk P Kingma, Tim Salimans, Rafal Jozefowicz, Xi Chen, Ilya Sutskever, and Max

Welling. Improved variational inference with inverse autoregressive flows. *Advances in NSural Information Processing systems*, 29, 2016.

Hirohisa Kishino, Jeffrey L Thorne, and William J Bruno. Performance of a divergence time estimation method under a probabilistic model of rate evolution. *Molecular Biology and Evolution*, 18(3):352–361, 2001.

Alp Kucukelbir, Dustin Tran, Rajesh Ranganath, Andrew Gelman, and David M Blei. Automatic differentiation variational inference. *The Journal of Machine Learning Research*, 18(1):430–474, 2017.

Mary K Kuhner, Jon Yamato, and Joseph Felsenstein. Estimating effective population size and mutation rate from sequence data using metropolis-hastings sampling. *Genetics*, 140 (4):1421–1430, 1995.

Ravin Kumar, Colin Carroll, Ari Hartikainen, and Osvaldo Martin. ArviZ a unified library for exploratory analysis of bayesian models in python. *Journal of Open Source Software*, 4(33):1143, 2019. doi: 10.21105/joss.01143. URL `https://doi.org/10.21105/joss.01143`.

David J Lunn, Andrew Thomas, Nicky Best, and David Spiegelhalter. WinBUGS - a Bayesian modelling framework: concepts, structure, and extensibility. *Statistics and Computing*, 10(4):325–337, 2000.

David JC MacKay et al. *Information theory, inference and learning algorithms*. Cambridge University Press, 2003.

Erick Matsen, Dave H Rich, Ognian Milanov, Mathieu Fourment, Seong-Hwan Jun, Hassan Nassif, Anna Kooperberg, Sho Kiami, Tanvi Ganapathy, Lucy Yang, and et al. `bito`. URL `https://github.com/phylovi/bito`.

Nicholas Metropolis, Arianna W Rosenbluth, Marshall N Rosenbluth, Augusta H Teller,

and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953.

Vladimir N Minin, Erik W Bloomquist, and Marc A Suchard. Smooth skyride through a rough skyline: Bayesian coalescent-based inference of population dynamics. *Molecular Biology and Evolution*, 25(7):1459–1471, 2008.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 2019.

Dan Piponi, Dave Moore, and Joshua V Dillon. Joint distributions for TensorFlow Probability. *arXiv preprint arXiv:2001.11819*, 2020.

Danilo Rezende and Shakir Mohamed. Variational inference with normalizing flows. In Francis Bach and David Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37, pages 1530–1538, Lille, France, 07–09 Jul 2015. PMLR.

Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, pages 400–407, 1951.

Fredrik Ronquist, Jan Kudlicka, Viktor Senderov, Johannes Borgström, Nicolas Lartillot, Daniel Lundén, Lawrence Murray, Thomas B Schön, and David Broman. Universal probabilistic programming offers a powerful approach to statistical phylogenetics. *Communications Biology*, 4(1):1–10, 2021.

John Salvatier, Thomas V Wiecki, and Christopher Fonnesbeck. Probabilistic programming in python using PyMC3. *PeerJ Computer Science*, 2:e55, 2016.

Tanja Stadler. On incomplete sampling under birth–death models and connections to the sampling-based coalescent. *Journal of theoretical biology*, 261(1):58–66, 2009.

Alexandros Stamatakis. RAxML version 8: a tool for phylogenetic analysis and post-analysis of large phylogenies. *Bioinformatics*, 30(9):1312–1313, 2014.

Marc A Suchard and Andrew Rambaut. Many-core algorithms for statistical phylogenetics. *Bioinformatics*, 25(11):1370–1376, 2009.

Christiaan Swanepoel. `phylojax`. URL `https://github.com/christiaanjs/phylojax`.

Simon Tavaré et al. Some probabilistic and statistical problems in the analysis of DNA sequences. *Lectures on Mathematics in the Life Sciences*, 17(2):57–86, 1986.

Jeffrey L Thorne, Hirohisa Kishino, and Ian S Painter. Estimating the rate of evolution of the rate of molecular evolution. *Molecular Biology and Evolution*, 15(12):1647–1657, 1998.

Thu-Hien To, Matthieu Jung, Samantha Lycett, and Olivier Gascuel. Fast dating using least-squares criteria and algorithms. *Systematic Biology*, 65(1):82–97, 2016.

Timothy G Vaughan, Denise Kühnert, Alex Popinga, David Welch, and Alexei J Drummond. Efficient Bayesian inference under the structured coalescent. *Bioinformatics*, 30(16):2272–2279, 2014.

Wangang Xie, Paul O Lewis, Yu Fan, Lynn Kuo, and Ming-Hui Chen. Improving marginal likelihood estimation for Bayesian phylogenetic model selection. *Systematic Biology*, 60 (2):150–160, 2011.

Ziheng Yang. Maximum likelihood phylogenetic estimation from DNA sequences with variable rates over sites: approximate methods. *Journal of Molecular Evolution*, 39(3): 306–314, 1994.

Ziheng Yang. PAML 4: phylogenetic analysis by maximum likelihood. *Molecular Biology and Evolution*, 24(8):1586–1591, 2007.

Ziheng Yang and Anne D Yoder. Estimation of the transition/transversion rate bias and species sampling. *Journal of Molecular Evolution*, 48(3):274–283, 1999.

Ziheng Yang, Rasmus Nielsen, Nick Goldman, and Anne-Mette Krabbe Pedersen. Codon-substitution models for heterogeneous selection pressure at amino acid sites. *Genetics*, 155(1):431–449, 2000.

Yuan Yu, Martín Abadi, Paul Barham, Eugene Brevdo, Mike Burrows, Andy Davis, Jeff Dean, Sanjay Ghemawat, Tim Harley, Peter Hawkins, et al. Dynamic control flow in large-scale machine learning. In *Proceedings of the Thirteenth EuroSys Conference*, pages 1–15, 2018.

Cheng Zhang. Improved variational Bayesian phylogenetic inference with normalizing flows. *Advances in Neural Information Processing Systems*, 33:18760–18771, 2020.

Cheng Zhang and Frederick A Matsen IV. Variational Bayesian phylogenetic inference. In *International Conference on Learning Representations*, 2018.

Cheng Zhang and Frederick A Matsen IV. A variational approach to Bayesian phylogenetic inference. *arXiv preprint arXiv:2204.07747*, 2022.

Rong Zhang and Alexei Drummond. Improving the performance of Bayesian phylogenetic inference under relaxed clock models. *BMC Evolutionary Biology*, 20:1–28, 2020.